

2-12-24

ADA lec no 15

Monday

Red Black tree:

Analysis: null = sentinel nodes = dummy head

Compare with BST

BST tree depends on order of input

RBT tree does not depend on order, same height for any order. it is search efficient.

RBT height: Running time $O(h)$
 $\{ \lg(n+1) \leq h \leq 2 \lg(n+1) \}$

BST height

$\{ \lg(n+1) \leq h \leq n \}$

→ BST traversal:

→ Euler Tour:

tree traversal can be found from Euler.

when you go neche from tree, write value of that

→ it is inorder.

→ left part = pre order

right part = post order

→ Hashing:

to search in constant time we have hash table which is it.

Ps. it was designed for dictionaries.

then extended to records. key ↑ satellite data
 we have key fields (id, name, sex)

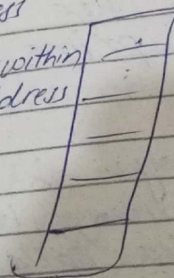
Key feature is search

Hashing two types: Hashing
 (i) open address chaining / Direct

(ii) Chain Hashing (open Hashing)

(i) Open address

hashing is closed within table but address open.



(ii) Hashing open, addresses are fixed.

(i) It has collisions, to fix we do linear probing (primary cluster) → quadratic (secondary cluster, cluster but after some time)

(ii) chain also called dynamic hashing
if data fixed, open address, if not
we chain hashing.

key words / reserved words, for its comp
we open address.

simple table, identifiers / variable
for its compiler, we chain hashing / probe

Modulus function:
to generate key, we take mod
for Direct, jitna value, utna ka
array but practically not possible

so we reduce size but use
modulus with prime function.

Hash fun:
to generate keys.
there can be many hash fun

① Modulus fun also called universal

$$h(K) = K \text{ mod } m$$

$h(K) = \text{Key} \cdot \frac{1}{\text{table size}}$
if ~~data~~, collision, do
linear probing

$$\text{index} = ((\text{Key} + \text{table size}) + i) \cdot \frac{1}{\text{table size}}$$

index
table [key] = key

Quadratic
 $\text{ind} = ((\text{Key} + \text{table size}) + i^2) \cdot \frac{1}{\text{table size}}$

$$\text{prob} = \frac{1}{\text{table size}}$$

→ Double hashing:
• rehashing / two hash functions

if table is full in open addressing, and
keys (dynamically) are not fixed.
we take double size hash table
at run time & insert those remain
keys in there.

which is better?

Mod method or Multiplication Method

→ Multiplication as in efficiency.
it use 2 processors.

for division (3 processors are used)
↓ expensive for processor.

→ Chained Hashing:

worst scenario is all keys in same chain.
running time is same as linked list
"practically its not always constant"

Open Address Hashing
table size = 07

0	21
1	
2	23
3	45
4	80
5	30
6	

linear probing

$$45 \div 7$$

$$21 \div 7$$

$$80 \div 7 \text{ (collision goes to 4)}$$

$$23 \div 7$$

$$30 \div 7 \text{ goes to 5 (4 probs)}$$

location ko dekho

80 (X) had to remove

search 30

because (4) is empty will not go to
so to avoid it we use additional flag
for deletion to use it as invalid.

all elements has ^{status} flag with it.

Analysis

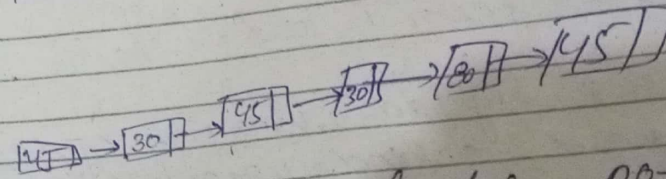
worst insert analysis

0	→ [21]
1	→ []
2	→ []
3	→ []
4	→ []
5	→ []
6	→ []

45, 21, 80, 23, front

we can insert at end. for that we have to traverse.

if we insert at front, don't need to traverse it.



its kharab as duplicates not allowed
so we use back push, so traverse
it. so that duplicates are not inserted.

before insert, we have to search.
insert is equal to search time.
insert prob = search prob.

Costs

C1 = insertion

C2 = compare cost

Total keys = n

C1 (1 + 2 + ... n-1) = $\frac{n(n-1)}{2}$ for avg
 "all keys in one chain"
 worst scenario = $\Theta(n^2)$

Best scenario

keys uniformly distribution
 if m keys & no of chains:

1 chain will have $\frac{m}{n}$ elements/keys

$$\alpha = \frac{m}{n}$$

→ load factor (load of chain)

jitna load zyada, utna prob zyada
 so search depend upon load factor
 chain dhondna

C1 to constant.

hash
fun
C2
↓
probing
cost

$$O(1 + \alpha)$$

if $\alpha = 0$, then running time is

insertion time is same as search time

Open Address Hash
 Running time $O(\frac{1}{1-\alpha})$

Chain Address $O(1 + \alpha)$

→ α = load on table in open Address

if table is 80% full, how avg how
 many probs it will enter in table
 $\alpha = \frac{80 \leftarrow \text{no of keys}}{100 \leftarrow \text{no of slots}} = \frac{m}{n}$

$$\frac{1}{1 - \frac{8}{10}} = \frac{5}{2} \text{ probs } \left\{ \begin{array}{l} \text{on the } \uparrow \\ \text{avg} \end{array} \right.$$

↓
insert & 5 Search

if 90% then probs will be 10 for
 inserting & searching

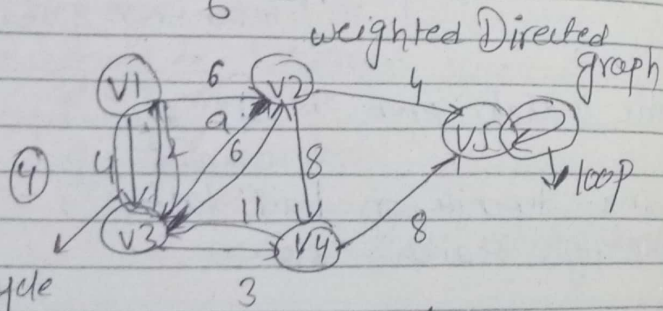
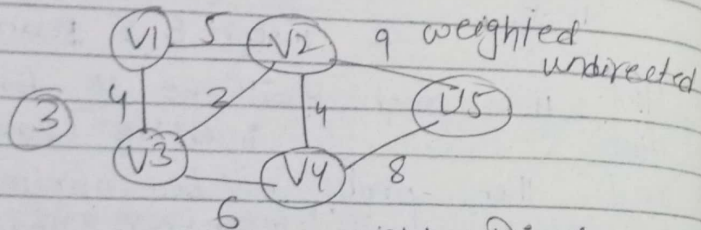
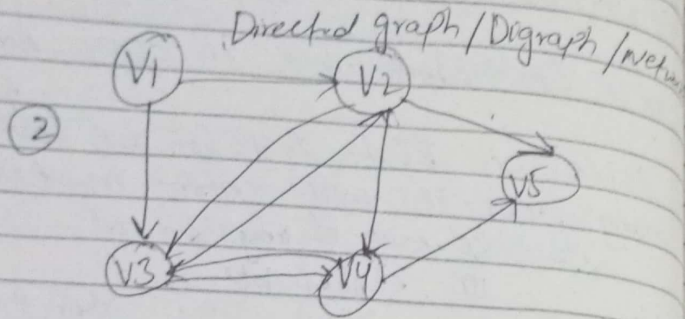
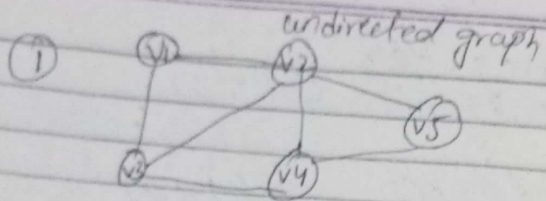
if 100% then probs will be infinite for
 inserting & searching

if table is khali, prob is 0

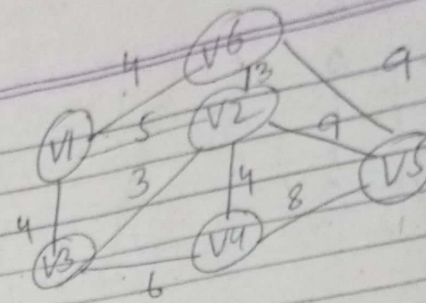
Hashing depends on load factor.

→ if table full, search slows.

→ Graph Algorithms:
 it is set of vertices & edges.



cycle
as it is cyclic graph, if cycle not exist
it is acyclic graph.



path to $V1 \rightarrow V4$

5

$V1$	$\rightarrow V2$	$\rightarrow V4$
$V1$	$\rightarrow V3$	$\rightarrow V4$
$V1$	$\rightarrow V2$	$\rightarrow V3 \rightarrow V4$
$V1$	$\rightarrow V3 \rightarrow V2$	$\rightarrow V4$

$V1 \rightarrow V2 \rightarrow V3 \rightarrow V2 \rightarrow V2 \rightarrow V4$

path for $V1 \rightarrow V5$
 $V1 \rightarrow V2 \rightarrow V5$

indegree (us ki taraf kitna edge)
outdegree (usai kitna jarahae)

when there are no vertices,
indegree = outdegree.

Graphical representation in matrix

	V_1	V_2	V_3	V_4	V_5
V_1	0	1	1	0	0
V_2	1	0	1	1	1
V_3	1	1	0	1	0
V_4	0	0	1	0	1
V_5	0	0	0	0	1

when edge exists = 1
when edge not exists = 0

we can draw graph from above
if diagonal has 1, it has loop

V_1 outdegree = 2 b/c
it has 2 1's

V_3 outdegree = 3

V_2 outdegree = 4

indegree = column "1's"
outdegree = row "1's"

"for Radix Sort, on every digit apply Counting Sort"

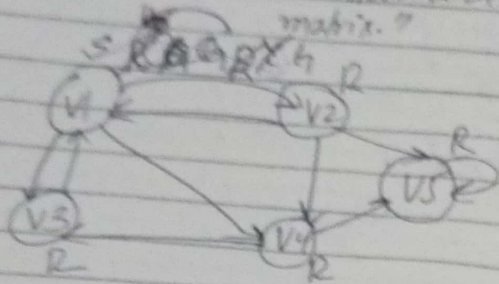
If weighted graph, write its weight.
If weight is distance & no edge on place, we will write ∞ .

4.12.2024

Graphs

Adjacency

matrix.
 evaluate based on input for



Matrix representation:

	V1	V2	V3	V4	V5
V1	0	1	1	1	0
V2	1	0	0	1	1
V3	1	0	0	0	0
V4	0	0	1	0	1
V5	0	0	0	0	1

→ list representation

1	→ 2	→ 3	→ 4	✓
2	→ 1	→ 4	→ 5	✓
3	→ 1	✓		
4	→ 1	→ 3	→ 5	✓
5	→ 4	✓		

advantages of matrix

easy to find in, out degree & path,
we find, shortest path, traversal done here
but mem acquire more.

in list / mem utilization is best.

→ Adj = it tells in & out degree

$$(Adj) \times (Adj) = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & 1 & 1 & 2 \\ 0 & 1 & 2 & 1 & 2 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

it tells 2 length path (indirect path involving 2 edges)

$$adj^3 = \begin{bmatrix} 2 & 0 & 1 & 1 & 2 \\ 0 & 1 & 2 & 1 & 2 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

jitna go utro pathu

	1	2	3	2	3
adj ³ v ₂	1	0	1	1	3
	2	0	1	1	2
	0	1	1	1	1
	0	0	0	0	1

v₂ → v₅

v₂ → v₅ → v₅ → v₅ } length=3

v₂ → v₄ → v₅ → v₅

v₂ → v₁ → v₄ → v₅

v₂ → v₁ → v₂ → v₅

→ Graph traversal:

→ 2 methods

(i) Depth, (ii) Breadth

(i) Depth first Search (DFS):

tell start vertex (on previous ex)
backtrack to v₅

v₁, v₂, v₄, v₃, v₅
"it is backtracking alg"

~~v₅~~ → neighbours (pop it)
goes to output v₁

every node color is red, only check red color
node goes to stack is yellow
all but node is green

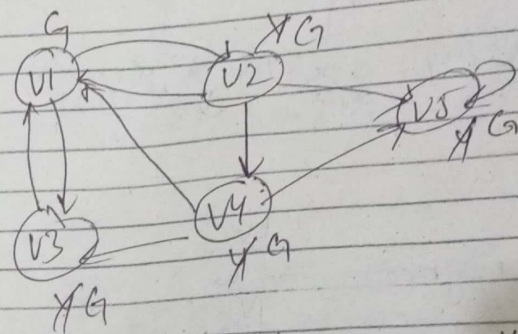
v₁ neighbour

v ₅
v ₃
v ₂

 v₄ neighbour

v ₅
v ₁

output v₁, v₄, v₅, v₃, v₂



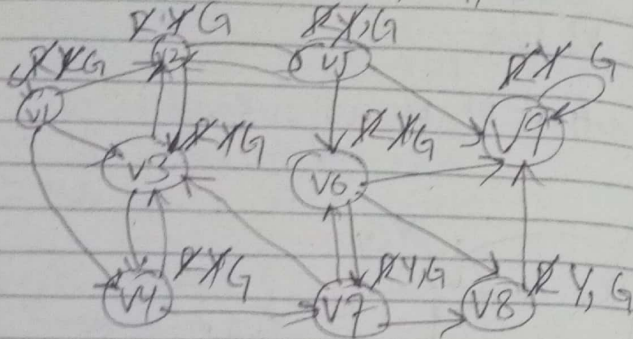
if any red still remains, it means at
that node of first that node is not traversed.
so purpose is dhondo wo node from
all are accessible

① Breadth first Search (BFS):

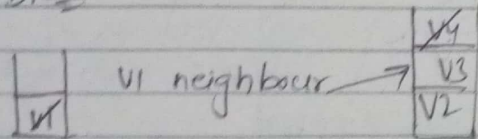
check v_1 neighbours & goes on, we use queue

$[v_1] \rightarrow v_2, v_3, v_4, v_5$ v_2 neighbour v_1 neighbour

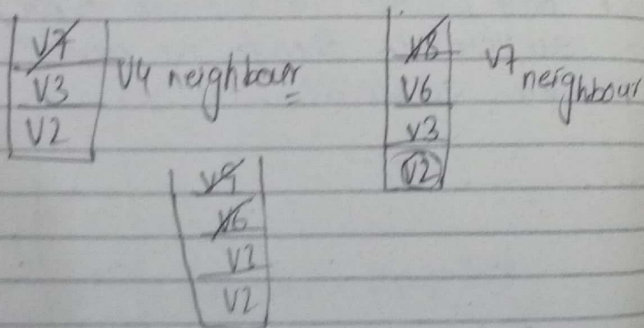
Output: v_1, v_2, v_3, v_4, v_5



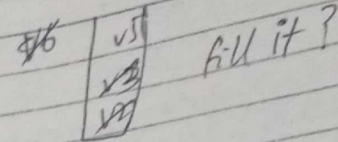
DFS



Output: $v_1, v_4, v_7, v_8, v_9, v_6, v_3, v_2, v_5$



for implementation
color 1 2 3 4 5
R 2 2 2 2 2
yes
G 0



Graph Alg:

$G = (V, E)$

$V = \{v_1, v_2, v_3, v_4, v_5\}$

Max possible edge $EC(V \times V)$ $\rightarrow$ max edge complete graph
if 5×5 matrix then max edge (25)

graph has vertices & edges sets.

$|V| = 5, |E| = 25$ $\leftarrow |E| = 25$
in actual graph, most prob $|E| < 25$

Dense complex $O(n^2)$

(sparse complex $O(n \log n)$) $\rightarrow$ list

$\rightarrow$ Paths:

undirected graph matrix is symmetric
 $[A^T = A]$, can use half matrix.

Memory space $O(V^2)$, $E = |V \times V|$

List Representation:

→ BFS:

Analysis

every node initialization = constant

$T_i = |V|$ = all vertex to be visited.

queue mai nikalna dalna time = (1)

all edges to be visited.

dalna ka time const

$$O(|V| + |E|)$$

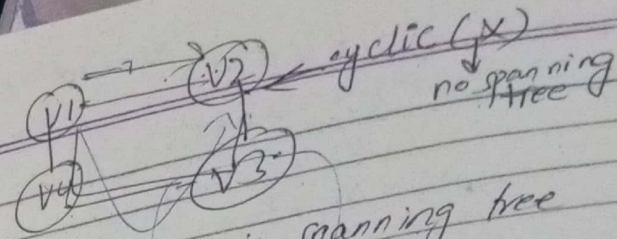
$2|V| \cdot E =$ dalna nikalna queue

→ DFS:

$$O(|V| + |E|) \rightarrow 2|V| \cdot O(1) = \text{path, pop}$$

→ Spanning Tree: of undirected graph subgraph which include all vertices of graph without ~~no~~ cycle, no multiple path (single) if multiple it becomes cycle.

blank edge is also sub-graph

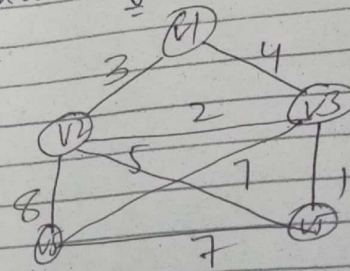


Result of BFS & DFS is spanning tree if all nodes visit.

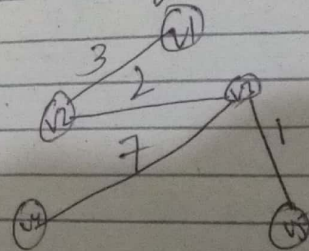
If graph is weighted, sum of edges weight, every spanning tree weight will be diff.

Min spanning tree is that which has min weight spanning that is optimization

→ Min spanning tree Alg:
Kruskal Alg



all edges must be removed.
& sort all edges, do it in priority queue



Sorted Edge

$$V3 V5 = 1 \checkmark$$

$$V2 V3 = 2 \checkmark$$

$$V1 V2 = 3 \checkmark$$

$$V1 V3 = 4 \times$$

$$V2 V5 = 5 \times$$

$$V3 V4 = 7 \checkmark$$

$$V4 V5 = 7 \times$$

$$V2 V4 = 8 \times$$

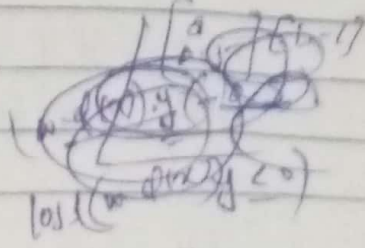
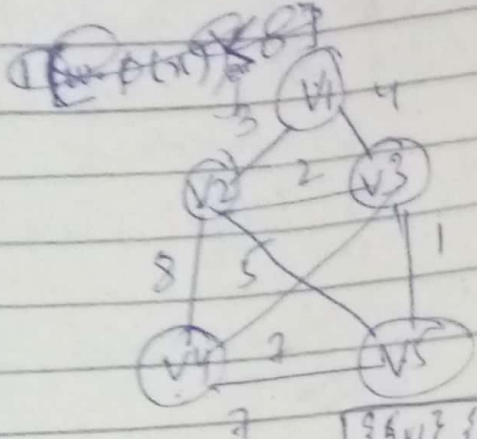
how to check cycles, will see union
set or set union called get of set.

11-12-2024

ADP Lec no

Wednesday

Set union:



priority queue	Edge	Weight	Decision
	V3-V5	1	add
	V2-V3	2	add
	V1-V2	3	add
	V1-V3	4	reject
	V2-V5	5	reject
	V3-V4	7	add
	V4-V5	7	reject
	V2-V4	8	reject

we want to find cycle for alg, use set union.

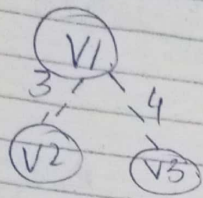
if both in one set = cycle
not 2 no cycle

it has found you min spanning tree.

V1 V2 V3 V4 V5 V6

Kruskal based on edges

Prim's Algorithm based on vertices

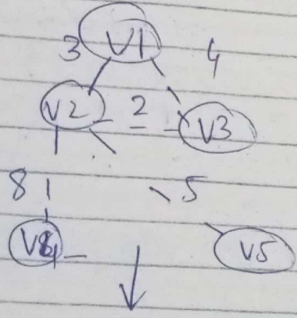


(V2 cost less selected that one)

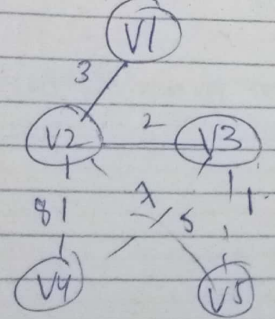
Need a start vertex (any) $\rightarrow$ if not given, start with shortest index (top of matrix)

$G = \{V1, V2, V3, V4, V5\}$
 $V = \{ \}$
 $\downarrow$ visited vertex
 $V = \{V1\}$
 $G = \{V2, V3, V4, V5\}$
 $V = \{V1\}$

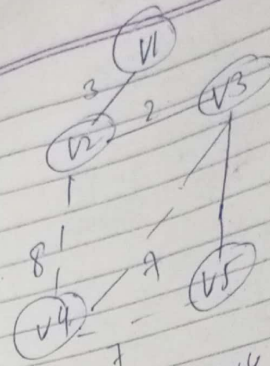
from every vertex to V to G , find edge.



$G = \{V3, V4, V5\}$
 $V = \{V1, V2\}$
 $(V3 \text{ less}) \rightarrow$ selected graph
 $G = \{V4, V5\}$
 $V = \{V1, V2, V3\}$
 (check all edges from all V to G)

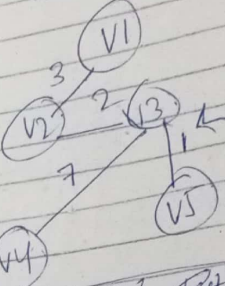


$G = \{V4\}$
 $V = \{V1, V2, V3, V5\}$



take $V3 \rightarrow V4$

$G = \{ \}$
 $V = \{V1, V2, V3, V5, V4\}$



minimum spanning tree

Finding Shortest path:

$\rightarrow$ Kruskal running time
 Priority is heap:

$$O(|E| \cdot \lg |E|) = O(E) = O(V^2)$$

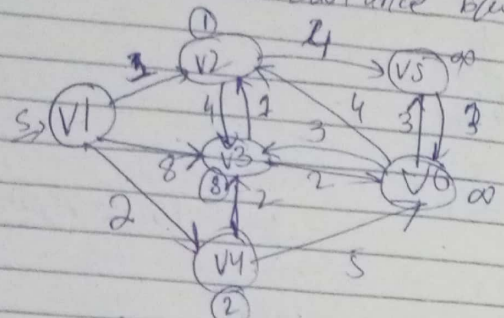
$$E \in O(|E| \cdot \lg |E|) \Rightarrow O(n^2 \lg n)$$

$\rightarrow$ Prim's Running time:

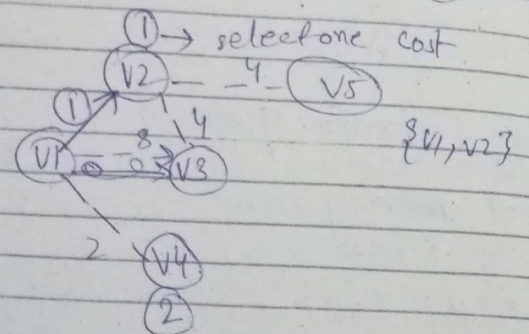
$G = \{ \}$ name diff in slides
 $V = V$ $\rightarrow O(n^2)$
 Prim's better than Kruskal

[8] [1] [9]
 → Shortest Distance: ~~Single source shortest distance~~
 → Single source shortest distance
 → Dijkstra's Alg

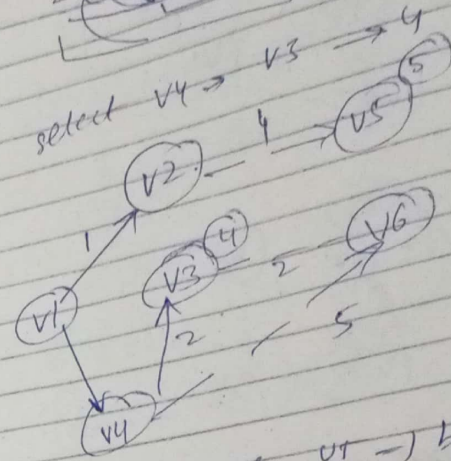
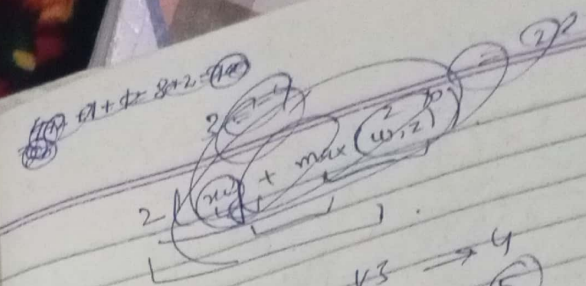
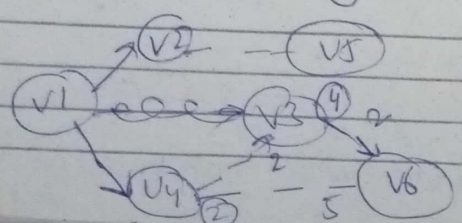
it is for directed weighted graph
 → find shortest distance b/w them



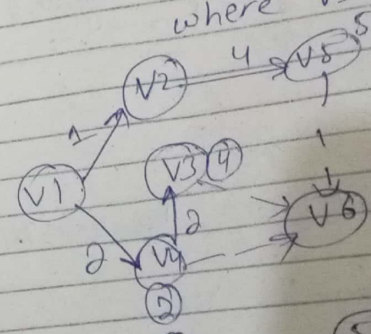
direct distance of no edge = ∞
 $V1 \rightarrow V6 \rightarrow \infty$ (8) direct



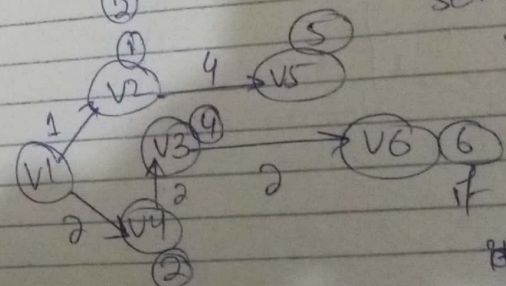
now select $V4$



we use $u \rightarrow v$ b/c it is 5
 where $v3 \rightarrow v6$ is 6

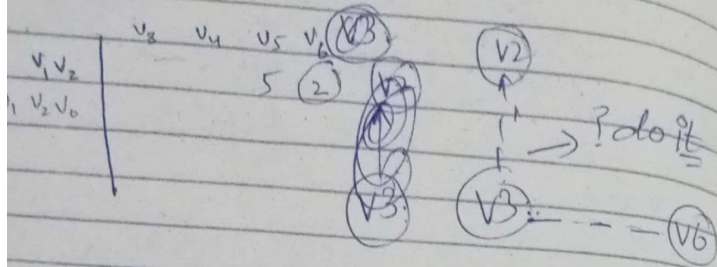
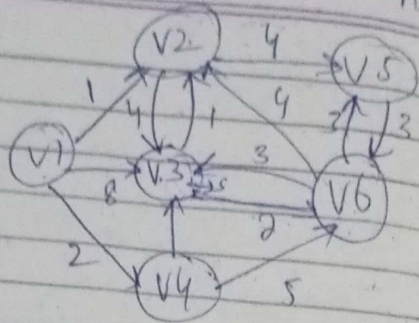


selected $V3 \rightarrow V6$

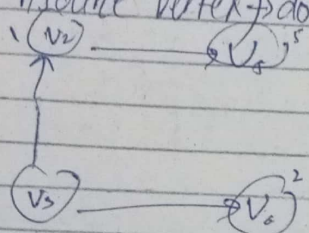


if $V5$ & $V6$ it becomes 8 whereas $V3$ to $V6$ becomes 6

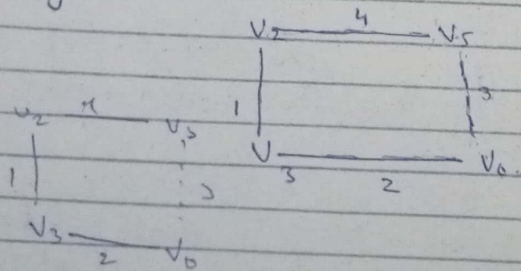
start state is V_3 now



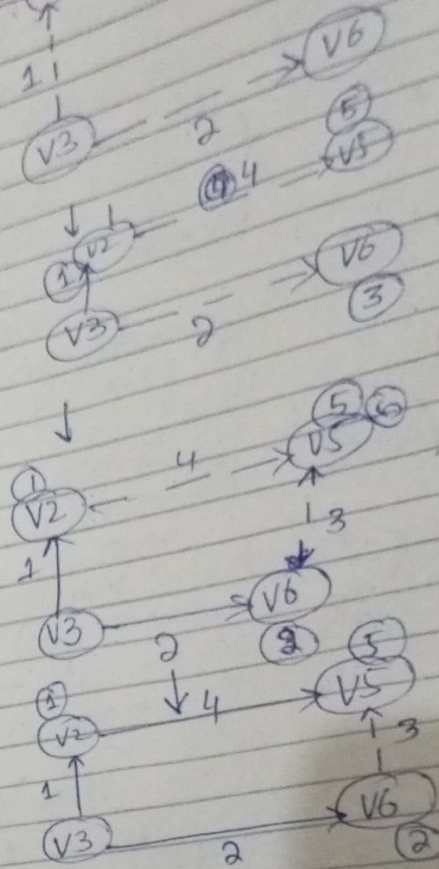
in slide (Arouse vertex) do it Algorithm 1.



"Implement algorithm", Dijkstra, Kruskal & Prim's



	V_1	V_2	V_3	V_4	V_5	V_6
V_1	0	1	8	2		
V_2	1	0	4			
V_3	8	4	0	3	2	2
V_4	2		3	0	5	5
V_5			2	5	0	3
V_6			2	5	3	0



18-12-24

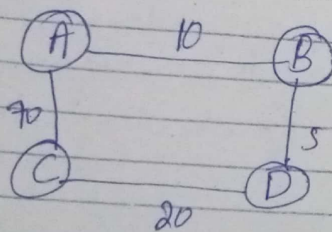
ADA Lec

Wednesday

→ Dynamic Programming:
technique for solving optimization prob.
alg works by splitting prob into
key of optimiz action sub prob
Main prob is global optimum
and sub are local optimum
& global made by local optm
problems

→ Principle of optimality:
key concept for DP.

- if local ^{sol} exists, then global exists. or
- if local ^{sol} _{becomes} part of global optm
sol, then principle of optimality
holds.
- if global exists but local ^{sol} _{does not}
part of global sol, so optimality
doesn't exist.

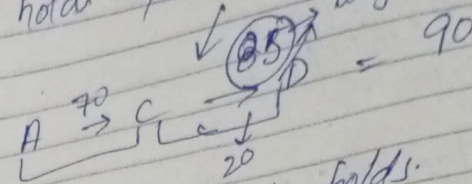


find A-D shortest distance.

$$A \xrightarrow{10} B \xrightarrow{5} D = \textcircled{15}$$

sub prob
 $A \xrightarrow{10} B$ & $B \xrightarrow{5} D$

local is part of global so principle
holds. find longest now



so DP principle not holds.
Advantages
solve opt prob very efficiently as
compare to brute force & in some
cases of greedy alg.

Disadv:

- memory overhead (solve bottom way
up) & in divide & conq (up-down)
sol stores & used same sol for same
overlapping prob.
- formulation of sub-prob is difficult.
why time more imp than space?
processor is mutually exclusive but
mem is shared resource so contextswitching
occurs which takes more time so
time more imp.

Applications:

(1) Fibonacci:

$$f(n) = f(n-1) + f(n-2)$$

$$f(0) = 1, f(1) = 1$$

	0	1	2	3	4	5	6	7
Fib	1	1	2	3	5	8	13	21

if $fib[0] = 1$
 $fib[1] = 1$
 for ($i = 2$ to n)

$fib[i] = fib[i-1] + fib[i-2]$
 end

above is DP sol.

Running time $\rightarrow O(n)$ Linear Recursive

Recursive sol $\rightarrow O(2^n)$

② Computing Binomial sol:
 nCr

③ Matrix chain Mult:
 provides min order to multiply any matrices
 Steps:

- i) identify structure of sub problem.
- ii) recursion recurrence of sub-prob.
- iii) set up recursion
- iv) bottom-up
- v) sol of sub-prob to global.

$\rightarrow$ ADG $\rightarrow$ Binomial Coeff:

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$$

$$(x+y)^n = \binom{n}{0} x^0 y^n + \binom{n}{1} x^1 y^{n-1} + \dots + \binom{n}{n} x^n y^0$$

$$c(n, k) = \frac{n!}{k!(n-k)!} \quad \text{OR } nCr = \frac{n!}{r!(n-r)!}$$

$$k=0 \text{ to } n \quad \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$(x+y)^{40}$ = factorial growth (very bad) have to find 40 coeff & have 40 factorial so very bad.
 it is very inefficient sol. for perm $P(n, k)$

Recursive Property:
 $c(n, k) = \binom{n}{k} = c(n-1, k) + c(n-1, k-1)$

Base case:
 $c(n, 0) = 1$ & $0! = 1$
 $\downarrow$ $nC0 = 1$

$nCn = c(n, n) = 1$
 $c(n, k) = 0$ if $k > n$ (case does not exist)

$\rightarrow$ Proof:

$$C(n, k) = C(n-1, k) + C(n-1, k-1)$$

take any sub-problem:

$$C(n-1, k) = \frac{(n-1)!}{k!(n-1-k)!}$$

sub prob kab main prob hai equal ho sakti
 $\Rightarrow C(n, k) = \frac{n!}{k!(n-k)!}$

$$C(n-1, k) = \frac{(n-1)! \cdot n}{k!(n-k)! \cdot n}$$

$$= \frac{(n-1)! \cdot n}{k!(n-k-1)! \cdot (n-k)}$$

$$\Rightarrow \frac{n(n-1)! (n-k)}{n k! (n-k)!}$$

$$= \frac{n! (n-k)}{n k! (n-k)!}$$

$$C(n-1, k) = C(n, k) \times \frac{(n-k)}{n} \Rightarrow (A)$$

Second sub pro

$$C(n-1, k-1) = \frac{(n-1)!}{(k-1)!(n-k)!} \times \frac{n}{k} \times k$$

$$= C(n, k) \times \frac{k}{n} \Rightarrow (C)$$

add eq (B) & (C)

$$= C(n, k) \times \frac{n-k}{n} + C(n, k) \times \frac{k}{n}$$

$$= C(n, k) \left[\frac{n-k}{n} + \frac{k}{n} \right]$$

$$= C(n, k) \left[\frac{n}{n} \right]$$

$C(n-1, k) + C(n-1, k-1) = C(n, k)$ proved

so eq (A) proved.

so

$$C[n, k] = C[n-1, k] + C[n-1, k-1]$$

two dimensional array.

find $C(6, 5)$
 $n=6$

$k \rightarrow$ b/c $k > n$

	0	1	2	3	4	5	6
0	1	0	0	0	0	0	0
1	1	2	1	0	0	0	0
2	1	3	6	1	0	0	0
3	1	4	10	10	1	0	0
4	1	5	15	20	15	1	0
5	1	6	21	30	21	6	1

$C[n, n] = 1$

↓ set here

$$C[2, 1] = C[1, 1] + C[1, 0]$$

$$C[3, 1] = C[2, 1] + C[2, 0]$$

if $(x+y)^6$ then 6 columns

so if we compute (6) we can get all previous coeff in $O(n^2)$

if $(x+y)^{40} \rightarrow 40 \times 40$ we get all coeff
 Running time $O(n^2)$
 Space Complexity $O(n^2)$

find $C(5, 4) = 5$
 $C(4, 3) = 0$

→ Matrix chain Multiplication:
 • first col & second row must be same. multiply is done by pehli ki row & dosri ki column.

$$C = A \times B$$

$$A = p \times q \quad B = q \times r$$

↓ same

$$A \times B = p \times r = C$$

$A_{1..2} = A_1 \times A_2 \times A_3$
 at a time multiply two matrix

$$A_1 = p_0 \times p_1, A_2 = p_1 \times p_2, A_3 = p_2 \times p_3$$

$$A_{1..3} = p_0 \times p_3 \rightarrow \text{dimension}$$

option 1

$$(A_1 \times A_2) \times A_3$$

option 2

$$A_1 \times (A_2 \times A_3)$$

select which order is best.

$$\begin{array}{ccc} A_1 & A_2 & A_3 \\ 4 \times 2 & 2 \times 7 & 7 \times 5 \end{array}$$

one more thing!

$$C = A \quad B$$

$$\begin{array}{cc} 4 \times 2 & 2 \times 7 \end{array}$$

Total no of mult operations = 56

$$\text{har row har column} = p \times q \times r = 4 \times 2 \times 7 = 56$$

option 1:

$$A_{1..3} = (A_1 \times A_2) A_3 = 56$$

$$\begin{array}{ccc} (56) \times & A_2 \times A_3 & \\ 4 \times 7 \times & 7 \times 5 & = 140 + 56 \\ & & = 196 \end{array}$$

option 2:

$$A_{1..3} = A_1 (A_2 \times A_3) = 70$$

$$\begin{array}{ccc} A_1 \times & A_2 \times A_3 & \\ 4 \times 2 \times & 2 \times 5 & \end{array}$$

$$A_{1..3} = 40 + 70 = 110$$

So in option 2, no of mult operations are less- so choose option 2.

Prob: we have chain of matrix, find optimum order of multiplication. Don't multiply it.

"it is called multiplication & addition operation".

20-12-24

ADA Lec

Friday

Matrix Chain Multiplications

seq $\rightarrow A_1 \cdot A_2 \cdot A_3$

dimensions $\rightarrow p_0 \cdot p_1 \cdot p_2 \dots p_{n-1} \cdot p_n$

Brute force: check all possibilities, which is best choice that

$n=10$
 $A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8 A_9 A_{10}$

for example split

$(A_1 A_2 A_3) (A_4 \dots A_{10})$

split again & again & goes to factorial $\Rightarrow$ called Catalan numbers

$$T(n) = \frac{(2n-1)!}{n(n-1)!!(n-1)!}$$

we start from start to closing

very bad.

Use bottom up approach acc to DP. start from base $\rightarrow$ split

$A_1 A_2 (A_3 \dots A_9) A_{10}$

find b/w $i \& j$ called k (where cost minimum) where $(i, j) = (k+1, j)$ if $i \leq k \& j \leq k$

if $i=1$ & $j=n$ then it is global problem.

Here all possibilities are explored but explored will not be explored.

if one matrix base case = 1
 if two matrix $\Rightarrow$ only one order save in table. $\} \rightarrow$ find minimum cost of 10 matrices by splitting

$A_1 A_2 \rightarrow \text{cost}$
 $A_2 A_3 \rightarrow 4$
 $A_3 A_4 \rightarrow 4$
 $A_4 A_5 \rightarrow 4$

now three three matrices

$(A_1 A_2 A_3) \Rightarrow (A_1 A_2) \cdot A_3 \& A_1 (A_2 \cdot A_3)$
 these are saved.

pehle 2 saved hai baqi ko add kar ka final cost now 4 matrices

3 orders
 pehla 4 second 4 aur 2

$(A_1 A_2 A_3 A_4) (A_5 \dots A_8) (A_9 A_{10})$

here 3 possible orders

$A_1 (A_2 A_3 A_4) \dots$ here two more
 $(A_1 A_2) (A_3 A_4) \dots$
 $(A_1 A_2 A_3) A_4 \dots$ two more
 total becomes 5 - $(A_1 \dots 3) (A_4)$
 mult op

$A_1 (A_2 A_3 A_4) \dots A_1 \cdot A_2 = 4$

Althri 2 waley kahi save nahi
have to add them
start split end
i k j

$i \leq k \leq j$
from i to (j-1)

$m[i, j] \rightarrow$ multiply all matrices
 $m[i, k] + m[k+1, j]$
 $\downarrow$
 A_{i-k} A_{k+1-j}

have to multiply last two matrices
so cost of these multiplication
will added which is their dimension

$\min_{i \leq k < j} (m[i, k] + m[k+1, j] + p_{i-1} \times p_k \times p_j)$
 $\downarrow$
 $p_{i-1} \times p_k$ $p_k \times p_j$
 $(p_{i-1}) p_j \times p_k$
multiplicative operations of final two matrices.
range $i \leq k < j$ (we need lesser value)

minimum with value of k.

$$m(i, i) = 0$$

matrix that save min optimum
multiplication operation & its two
dimension so contains 2 dimension
in final.

dimensions array
p 0 1 2 3 4 5
4 2 7 5 3 6

A_1 A_2 A_3 A_4 A_5
4x2 2x7 7x5 5x3 3x6
 $A_1 A_2 A_3$

	A_1	A_2	A_3	A_4	A_5
A_1	0	56	112	124	136
A_2	0	0	70	105	231
A_3	0	0	0	90	0
A_4	0	0	0	0	0
A_5	0	0	0	0	0

using formulae
 $1 \times 2 = 2$ $2 \times 1 = 2$
(nothing)
calculations on diagonal
 $m[1, 2] = \min(m[1, 1] + m[2, 2] + p_0 \times p_1 \times p_2)$
 $m[1, 2] = \min(0 + 0 + 4 \times 2 \times 7) = 56$

$i \leq k < j$
 $i=1, j=2, k=1$

$m[1, 3]$ $k=2$
when $k=1$ $0 + 70 + 40 = 110$
 $m[1, 3] = m[1, 1] + m[2, 3] + p_0 \times p_1 \times p_3$
when $k=2$ $56 + 0 + 140 = 196$
 $m[1, 3] = m[1, 2] + m[3, 3] + p_0 \times p_2 \times p_3$

$m[2, 4]$ $k=3$
when $k=2$ $196 + 90 + 231 = 517$
 $m[2, 4] = m[2, 2] + m[3, 4] + p_1 \times p_2 \times p_4$
when $k=3$ $70 + 105 + 90 = 265$
 $m[2, 4] = m[2, 3] + m[4, 4] + p_1 \times p_3 \times p_4$

$m[1, 4] = m[1, 1] + m[2, 4] + p_0 \times p_1 \times p_4 = 124$
 $k=1$
 $k=2$
 $k=3$
 $m[1, 4] = m[1, 2] + m[3, 4] + p_0 \times p_2 \times p_4 = ?$

$$m[1,4] = m[1,3] + m[4,4] + p_0 \times p_3 \times p_4 = 170$$

How to know where to split
make another matrix $\rightarrow$ use values of k !

	A_1	A_2	A_3	A_4	A_5
A_1	-	1	1	1	1
A_2	-	-	2	3	4
A_3	-	-	-	3	4
A_4	-	-	-	-	4
A_5	-	-	-	-	-

$\neq 0$ order matrix

~~6~~ 6×6 matrix
personally.

$$k=1 \quad m[1,5] = m[1,1] + m[2,5] + p_0 \times p_1 \times p_5 = 184$$

$$k=2 \quad m[1,5] = m[1,2] + m[3,5] + p_0 \times p_2 \times p_5 = 288$$

$$k=3 \quad m[1,5] = m[1,3] + m[4,5] + p_0 \times p_3 \times p_5 = 120$$

$$k=4 \quad m[1,5] = m[1,4] + m[5,5] + p_0 \times p_4 \times p_5 = 176$$

optimum min operations = 184 & $k=1$

$$(A_1 ((A_2 A_3) A_4) A_5)$$

$$\# \{2 \dots 5\} = 4$$

$$\# \{2 \dots 4\} = 3$$

DP Running $O(n^3)$

Brute force $O(\text{factorial}) = (2n-1)!$

DP produce efficient results as compare to Brute force

6-1-25

→ DP Application

→ Longest common subsequence (LCS):

sub sequence = set of char in proper order

String: substring = set of char

SI = "XAB YND"

Zarori nahi wo akathe hon

Monday

akathe hon

6-1-25

→ DP Application

→ Longest common subsequence (LCS):

sub sequence = set of char in proper order

String: substring = set of char

SI = "XAB YND"

Zarori nahi wo akathe hon

Monday

akathe hon

subsequence: set of char in proper order
String: substring: set of char all the time

SI = "XAB YND"

$S1 = \text{"XAB YND"}$

$S_2 = \text{"MBAXYCD"}$

51 sub string = ABV

sub sequence $\rightarrow$ ABY
 AYD, XY, D, YD
 not equal $\rightarrow \neq DA, YA$
 XYD
 ~~XYD~~
 $a \rightarrow$ empty string

har sub-string sub seq but har sub-is
sub-string nahi."

common sub-seq

SL, 32 = X, Y, D → XYD
AYD
BYD
YD
XD
D
→ empty string

→ through Brute force:
check all possibilities (bruteforce search)
 $s1 = n$ length
 $s2 = m$ length

find all possible sub-seq of string &
compare it on linear search, jaha
match hoga woh LCS
if n then possible sub-seq (2^n)
2. 2^n (A, B, AB, ^)

if n is odd then $\frac{1}{2}n$ is not an integer
 $AB \rightarrow 2^2 = 4^2 (A, B, AB, \dots)$

AB →
 $O(m \cdot 2^n)$ → Exponential string search karega.
 "all mai find karega derhona search karega."

By $O(n \times m)$

By DP
 (*) $O(n \times m)$
 • Matrix has n rows & m col.
 • bottom up - way (sab se choti prob ko
 sab se badi problem).
 Definition:

application:
DNA matching

→ example

example
 $S_1 = ABAC$
 $S_2 = BADB$

$S_2 =$

	$\wedge$	A	B	$\neg A$	C
$\wedge$	0	0	0	0	0
B	0	0	1	1	1
A	0	1	1	2	2
D	0	1	1	2	2
B	0	1	2	2	2
C	0	1	2	3	3

if $i \in j = 0$
no string part
length of common
subseq = 0

ager match mahi, upper aur
left side par ha wo
use karo

agar char match ^{nahi} kar gaya preu length
sub seq - this sub seq
agar match kar gaya matching ho dono sai nikal
nai, pehla wale length me +1 kar do.

for seq

	A	B	C
B	B	B	A
D	A	D	L
D	D	B	D
B	B	U	L
C	A	U	U

yaha ^{sa}shuru

BAC

ma in stack.

→ Example 2: DNA Matching:

→ * Huffman Coding (By David Huffman)

text mai jin ki freq zyada hai
un ka no of bits kam, jin ki freq
kam hai, un ka no of bits zyada.

if $A = 30$ times, $X = 2$ times
if $A = 1$ bit, then 30 bits.

tree

examp/c:

A, B, C, A, A, B, A

A = 6

B = 3

C = 2

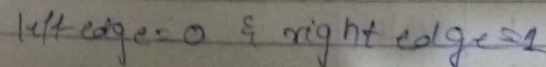
D = 1

	D	C	B	A
A	6			
B	3			
C	2			
D	1			

tree nary, in

D^1 C^2 B^3 A^6

take out 2, D & C
 (3) → adding freq of D & C
 (D) (C) | B³ A⁶ } gone in queue again
 now B & 3 same, so remove



L	F	Bit Pattern
A	6	0
B	3	10
C	2	111
D	1	110

→ binary

decrypt → A B A C A A B A D C B A

0 10 0 111 0 0 10 0 110 111 10 0

A B A C A A B A D C B A

relieving end par bit pattern aya

key is free.

0 means left, 1 right

A B A C A A B A D C B A

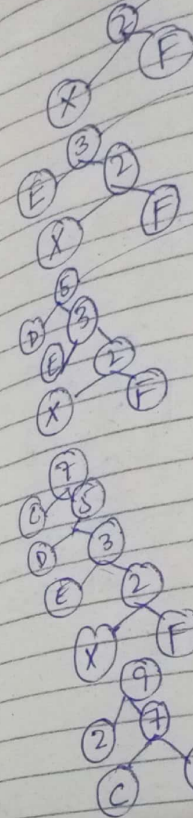
for decryption →

→ Main purpose is compression.

• for decryption, we required tree.

Example 2:

L	F	Bit Pattern
A	25	1
B	10	01
C	8	0001
D	2	0010
E	1	0000
F	1	0001
X	1	0010



9

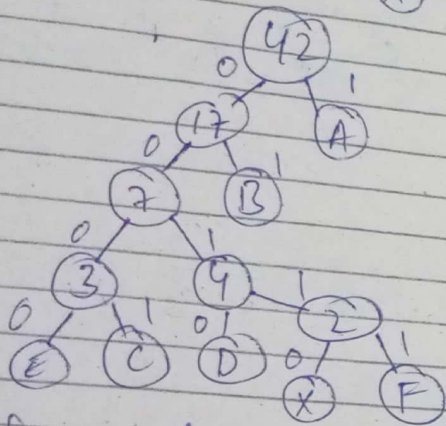
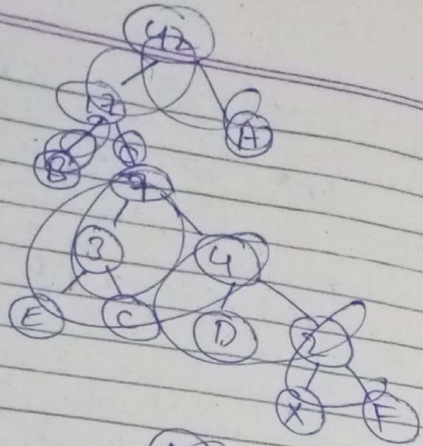
X' F' E' D' C' B' A'

E' D' C' B' A

D' C' 2 A B' A'

C' 2 3 A B' A'

2 3 B A



char freq

1	1
C	F

leaf mai
domo ho ga
non leaf mai char
ki jagah dummy
rakh dena.

for freq finding,
make hash table

jaha char mila us ki position mai ja kar
kar do

leaf mai char & freq dono hai &
non leaf mai char nahi hai.

one node has char & freq
take var bit, traverse the tree &
you have bit pattern.

bit = 0000 → when char
occurs assign that value of bit
to that char.

one step previous, one minus bit
bit = 00010 → char = E,
all tree traverse

input text file, read, har, hash
table, tree, freq calculate, and in
another file, print bit pattern.

now for decryption, give encrypted file
and it will print string (original one)
in another file.

8-1-25

ADA Lec

Wednesday

String Processing Algorithms:

- Categories:
 - Data Compression / String
 - String Matching

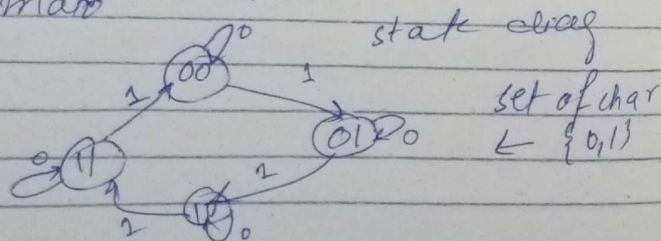
① Data Compression:
alg is Huffman Code,
→ zip file made by LZW.

Heap insert $\lg(n)$
delete $\lg(n)$

So it becomes $3 \times \lg(n)$
 $O(n \lg n) \rightarrow$ tree creation
tree traverse $\rightarrow O(n)$
So $O(n \lg n + n) \rightarrow O(n \lg n)$

→ String Matching Alg:
it is brute force (linear search of text). (naive string matching)

• finite State automaton:
state diag in seq circuits in DLD is automata

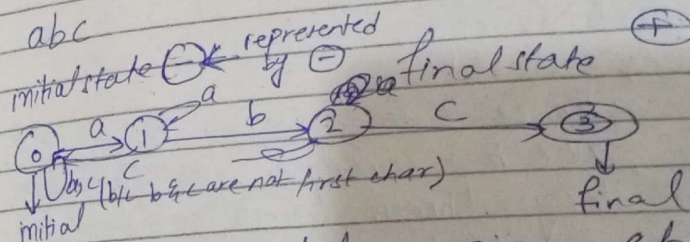


state table

	0	1
00	00	01
01	01	10
10	10	11
11	11	00

in automata state moving are transitions
→ search in text is pattern.

text 2 jai mai -ai search
pattern 2 jai ai search, is ka automata
bane ga.



two representations given above
→ is transition

③ is termination state or called acceptance state.

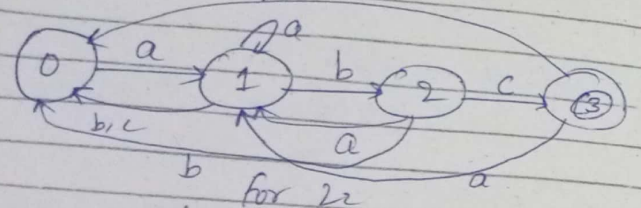
har state ki transition power karo

set of char rep by $\Sigma \{a, b, c\}$
har state per 3 trans b/c of 3 char

	a	b	c
0	1	0	0
1	1	2	0
2	1	0	3
3	1	0	0

pattern kharab ho gaya
waps job

If on 1, a agains agaya { do 3
remove previous & go with next



for 2
a b a → start with a
a b b → wrong
for 3
a b c a → for again pattern
a b c b → wrong, a b c c → wrong

→ Naive string: (linear search)
jab pattern mismatch hua shuru se
wile step agai jao.

Running time.

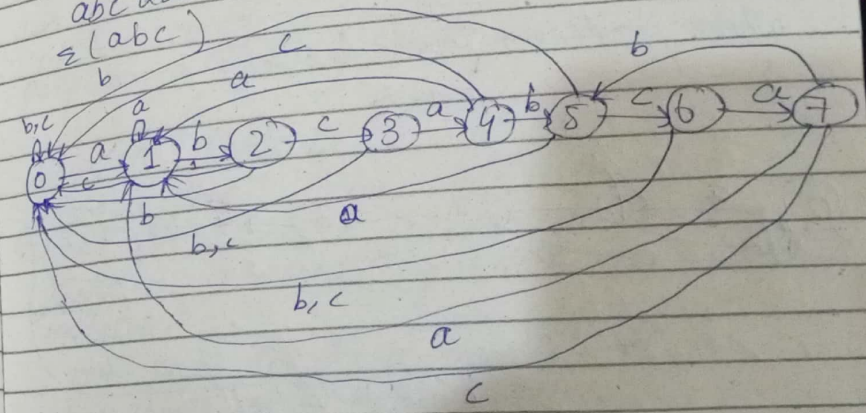
$O(mn)$
first loop - $n - m + 1$
 n = whole char, m = pattern char
second loop $1 \rightarrow m$

$$O(m(n-m+1)) \in O(mn)$$

→ finite state Automaton:
for string pattern matching

$Q \rightarrow$ set of states
In previous ex $Q = 4$
 $q_0 = 0, q_f = 3$
 $q_0 \in Q, q_f \in Q$
 $\delta \rightarrow$ transitions.

Ex no 2
abc abc a
 $\delta(abc)$



for 3 → abc a
abc b → wrong
for 4, abc a → wrong gone to 1
for 6, abc abb

see in pattern if there is a overlapping
pattern, see if that it can start
from it again.

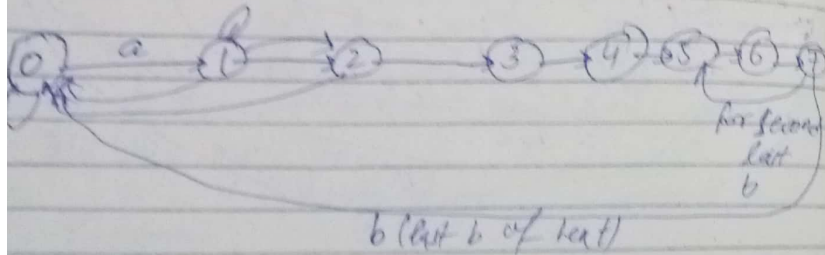
Transition Table:

	a	b	c
0	1	0	0
1	1	0	0
2	1	0	3
3	4	0	0
4	1	5	0
5	1	0	6
6	7	0	0
7	1	5	0

search in this { above is binary automata }

aabbabcabcabcabb

when pattern never lapping
state



execution of automaton (fa)

Last topic: chap 9 from book.
theory of NP completeness:
Computational Complexity & Intractability
and in impact - we have are
category of NP. so we study that.
→ sorting is prob & sol is quick sort.

this theory deals with prob not alg
on basis of alg complexity.
Comp complexity → complexity of Alg.
we go on basis of alg complexity
sol are categorized as:
polynomial & super polynomial.

Now in Problem:
interactivity how diff to solve
prob for comp? what difficulties?
that is intractability (difficult to treat
or work).
any prob that is must be to solve
for comp is intractability.
like 40 node sol of hamiltonian

prob that are easy to solve for
comp is tractable.
prob that have polynomial sol is
tractable and prob that have
no polynomial sol is intractable.

sol exist (polynomial)
if sol don't exist (non-polynomial)
sorting $\rightarrow$ tractable.

Counting based sorts $\rightarrow$ tractable b/c they have sol.
Fibonacci is tractable b/c they have iterative sol.

interactable has two categories:
so overall they have 3 categories

(i) problems jin ka polynomial sol hai hi nahi (proved) - mat dhondo

(ii) prob jin ka prove nahi hua e sol mila bhi nahi hai. (focus area of research)

find min spanning tree (sol are known & prim) before this they were intractable but now they are tractable.

(iii) jin ka sol mil gaya from (ii) wo tractable

$\rightarrow$ (i) jin ko no sol (proved)
find all hamiltonian circuit prob.
it is proven intractable.

(ii) + find all ole tour.

(ii) find a hamiltonian circuit in a graph (prove nahi hua)
find ole tour

(i) $\rightarrow$ these are called hard problems (intractable prob).

40 nodes network, how many possible networks of hamiltonian.
39 or 28!

if n nodes graph then above are $n!$ hamiltonian circuit
u approx takes

if 1 circuit takes 1 min then all are $n! \times 1$

if results are super polynomial then they can't be in polynomial, so it is given as intractable.

another example is halting problem.

Not Exhaustive testing

if there is int range 0-999

don't put all values, do enter 20% more than 999, that is boundary checking, don't do exhaustive.

if input is super polynomial, then
they halt process in polynomial,
exhaustive is in which input process
is halt.

15-1-25

ADA ke

Wednesday

decision prob & ans yes or no
optimization prob.

max prob, min prob, shortest
longest

optimization can be converted to
decision prob →

like ^{is there} Hamiltonian circuit?

Yes or No (converted to Decision
sol finding same.

- is graph is 3 color able or not?
Yes or No.

find min
- spanning tree in graph (optim
prob)

Is there any span tree less
than 3 grade? converted to Decision
has optimize converted to Decision

→ Verifiability:

hard prob sol finding difficult
so ham sol each dete, we guess the
sol, then verify it. agar guess sol
hai to ~~that~~ ^{sol mil gya} nahi to ~~that~~ ^{ko aur}

guess karo.

- Sorting example, we guess sol
for it. 5, 10, 20, 30
is this sorted sol?

we check it, agla no barha hai
ya nahi. so we have alg.

agar agla barha hai to theore,
 $O(n^2)$ linear.

Kia ham verify polynomial time kar
sakte ya nahi? agar yes to okay

$V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow V_4 \rightarrow V_5 \rightarrow V_1$

verify is it hamiltonian

pehle or akhri same ✓

length or node same hone chaye

also see vertex repeated or not.

is ka poly sol nahi but guessed
sol poly mai ho raho.

→ N-P theory based on decision problem
first class. (in classes, we have Decision prob)

P-class

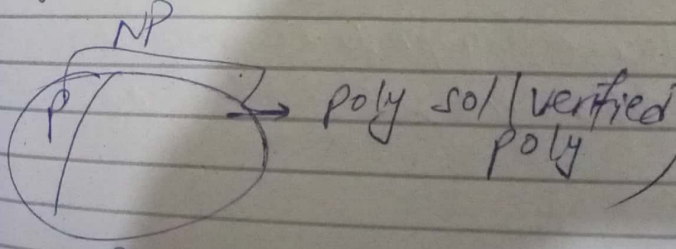
guessed or prepared sol ~~can~~ poly

they are tractable & easy

P-class → ^{classes - decision} tractable

NP → poly sol hai ya nahi

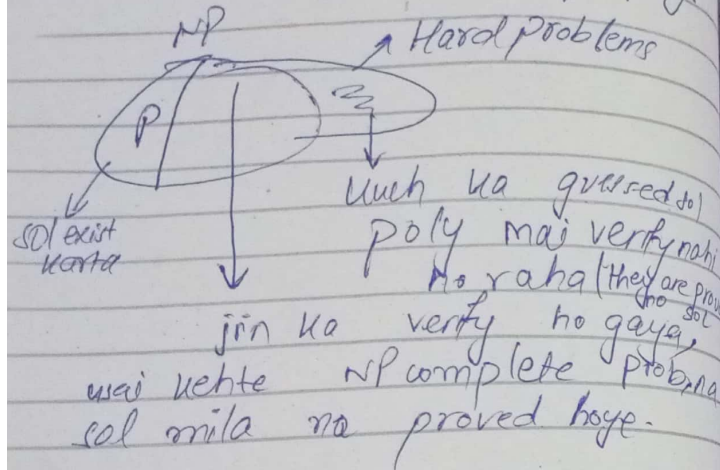
→ guess kar sakte decision prob. ↳ super poly



NP - super set

P - subset of poly

NP-complete sets
 prob which are hard (poly sol may exist) but any guessed or proposed sol must be verified as guess poly.

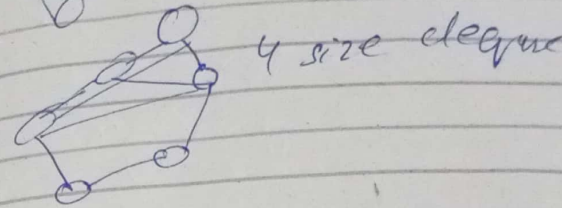
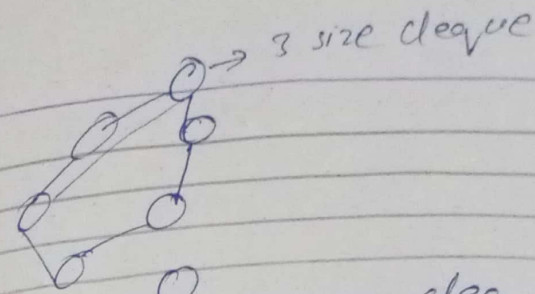


Hard mai (All halting problem, etc)

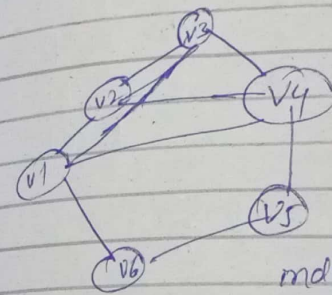
examples:

- graph coloring
- SAT prob (satisfiability prob)
- CNF-SAT (conjunction normal form SAT)
- 3-CNF-SAT (3 literal values)
- imp problem (NP-complete linked)

- clique (graph ka aisa sub graph jo comparsa mai har node linked hai)
- independent set



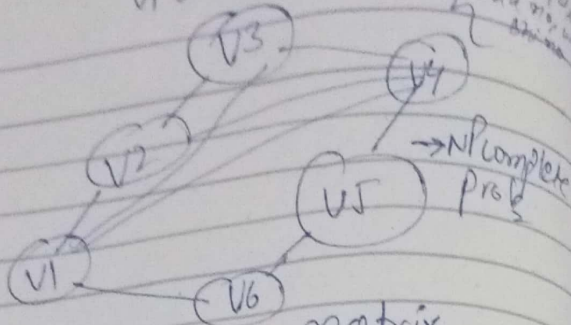
So find max size clique (optim. prob)



2 sai barha link nahi

NP complete prob mai agar kisi ka sol mil gaya to bagis ka bhi mil jae ga, b/c all of are linked."

degree max size
 $V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow V_4$
 go sol in 4
 40 complete
 10 for 40
 30 for 10
 10 for 30



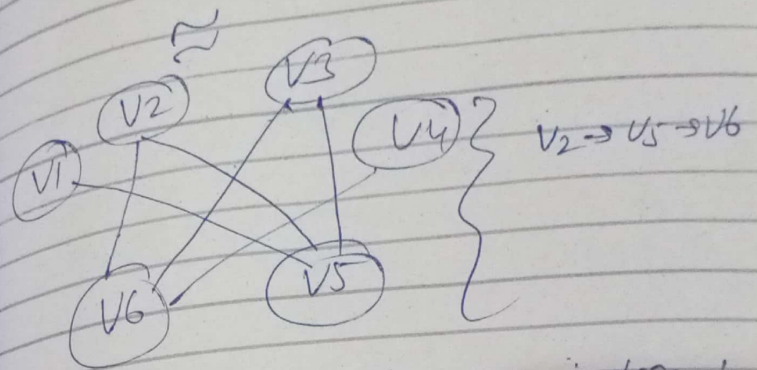
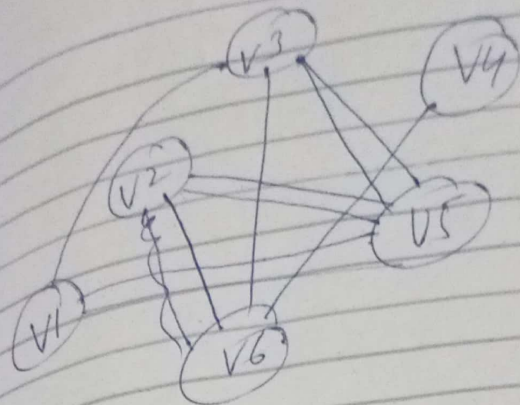
draw its adjacency matrix

	V1	V2	V3	V4	V5	V6
V1	1	1	1	1	1	1
V2	1	1	1	1	1	1
V3	1	1	1	1	1	1
V4	1	1	1	1	1	1
V5	1	1	1	1	1	1
V6	1	1	1	1	1	1

diagonal ko chor hai baqi sab koulke adjacent

	V1	V2	V3	V4	V5	V6
V1	0	0	0	0	1	0
V2	0	0	0	0	1	1
V3	1	0	0	0	0	0
V4	0	0	0	0	0	0
V5	1	1	1	0	0	0
V6	0	1	1	1	0	0

now draw matrix



$V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow V_4$ are independent

degree ko poly mai change kar lia, inde so it is a NP-complete problem.

above
 so it is done through SCNF - (AT 6)
 SCNF - (AT 6)

so agar koi aik prob of NP solve ho gai to sarey NP solve ho jayen ge
 ki there is reduction (reduction means jo waha degree they wo waha indep ho gaye).